SAVEETHA SCHOOL OF ENGINEERING

C PROGRAMMING ASSIGNMENT

Comprehensive System Design and Implementation Challenge

Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Comprehensive System Design and Implementation Challenge – District Blood Bank Network
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	13 NOVEMBER 2026

TASK B1 — Implement and Instrument the Recursive Functions (7 Marks)

Task / Problem Statement

Implement the digital root and both power functions, test them, count the recursive calls for n = 10, 100, 1,000 and 10,000, and find the value of n at which linear recursion crashes.

Aim

To verify the recursive designs of A2 and to show with measurements why halving is better than linear recursion and why recursion depth is limited.

Concept / Theory

Each recursive call creates a stack frame. Linear recursion makes n + 1 calls and needs n frames; halving recursion makes about $\log_2 n + 2$ calls. The stack is limited (about 8 MB), so a deep recursion ends in a stack overflow. The compiler is run with -O0 so that it does not turn the recursion into a loop.

Procedure / Algorithm

1. Write digit_sum, digital_root, power_linear, power_fast and an iterative power_iter (pseudocode of A2.1).
2. Add a global counter calls that every power function increments on entry.
3. Test with ordinary values, 0, 1 and one value of my own choice and compare the three power versions and the formula.
4. For n = 10, 100, 1,000, 10,000 reset the counter, run each version, and print the counts (base = 1 so the result never overflows).
5. Complete TODO 1 of the given program, run it, then change half * half to two calls and count again.
6. Run the linear version with growing n until it crashes (bisection); repeat with the iterative version.

Program / Source Code

Program 1 – b1_recursion.c (tests and call counts):

```
#include <stdio.h>

long long calls = 0;                    /* number of recursive calls made */

/* ---------- digital root ---------- */
int digit_sum(long n)                    /* recursive sum of digits */
{
   if (n < 10) return (int)n;           /* base case: one digit */
   return (int)(n % 10) + digit_sum(n / 10);
}
int digital_root(long n)
{
   if (n < 10) return (int)n;           /* base case: already one digit */
   return digital_root(digit_sum(n));    /* recursive case */
}
int digital_root_formula(long n) { return n == 0 ? 0 : 1 + (int)((n - 1) % 9); }

/* ---------- power ---------- */
long long power_linear(long long base, int exp)
{
   calls++;
   if (exp == 0) return 1;
   return base * power_linear(base, exp - 1);
}
long long power_fast(long long base, int exp)
{
   calls++;
   if (exp == 0) return 1;                          /* base case */
   long long half = power_fast(base, exp / 2);          /* computed ONCE */
   if (exp % 2 == 0) return half * half;
   return half * half * base;
}
long long power_fast_twice(long long base, int exp)       /* the "wrong" version */
{
   calls++;
   if (exp == 0) return 1;
   long long r = power_fast_twice(base, exp / 2) * power_fast_twice(base, exp / 2);
   return (exp % 2 == 0) ? r : r * base;
}
long long power_iter(long long base, int exp)            /* square-and-multiply */
{
   long long result = 1;
   while (exp > 0) {
      if (exp & 1) result *= base;
      base *= base;
      exp >>= 1;
   }
   return result;
}

int main(void)
{
   puts("== Test table: digital root ==");
   long dr_in[] = {0, 1, 9, 38, 9875, 99999, 493193};
   for (int i = 0; i < 7; i++)
      printf("digital_root(%ld) = %d   formula = %d\n", dr_in[i], digital_root(dr_in[i]), digital_root_formula(dr_in[i]));

   puts("\n== Test table: power ==");
   long long tb[][2] = {{2,10},{5,0},{7,1},{3,13},{0,5},{1,1000}};
```

```
    for (int i = 0; i < 6; i++) {
        long long b = tb[i][0]; int e = (int)tb[i][1];
        printf("%lld^%d: linear=%lld fast=%lld iter=%lld\n", b, e, power_linear(b,e), power_fast(b,e), power_iter(b,e));
    }

    puts("\n== Recursive call counts (base = 1 so values never overflow) ==");
    printf("%8s %10s %8s %12s\n", "n", "linear", "fast", "fast-twice");
    int ns[] = {10, 100, 1000, 10000};
    for (int i = 0; i < 4; i++) {
        long long cl, cf, ct;
        calls = 0; power_linear(1, ns[i]);     cl = calls;
        calls = 0; power_fast(1, ns[i]);       cf = calls;
        calls = 0; power_fast_twice(1, ns[i]); ct = calls;
        printf("%8d %10lld %8lld ", ns[i], cl, cf);
        if (ct >= 0) printf("%12lld\n", ct); else printf("%12s\n", "(see note)");
    }
    calls = 0; power_fast_twice(1, 30);
    printf("\nfast-twice for n = 30: %lld calls\n", calls);
    return 0;
}
```

Program 2 – the given program with TODO 1 completed (b1_todo.c):

```
#include <stdio.h>

long long calls = 0; /* number of recursive calls made */

/* Linear recursion: base^exp = base * base^(exp-1) */
long long power_linear(long long base, int exp)
{
    calls++;
    if (exp == 0) return 1;
    return base * power_linear(base, exp - 1);
}

/* Divide-and-conquer recursion: compute base^(exp/2) once and square it. */
long long power_fast(long long base, int exp)
{
    calls++;
    if (exp == 0) return 1;                    /* base case */
    long long half = power_fast(base, exp / 2);  /* computed ONCE */
    if (exp % 2 == 0) return half * half;        /* even exponent */
    return half * half * base;                   /* odd exponent  */
}

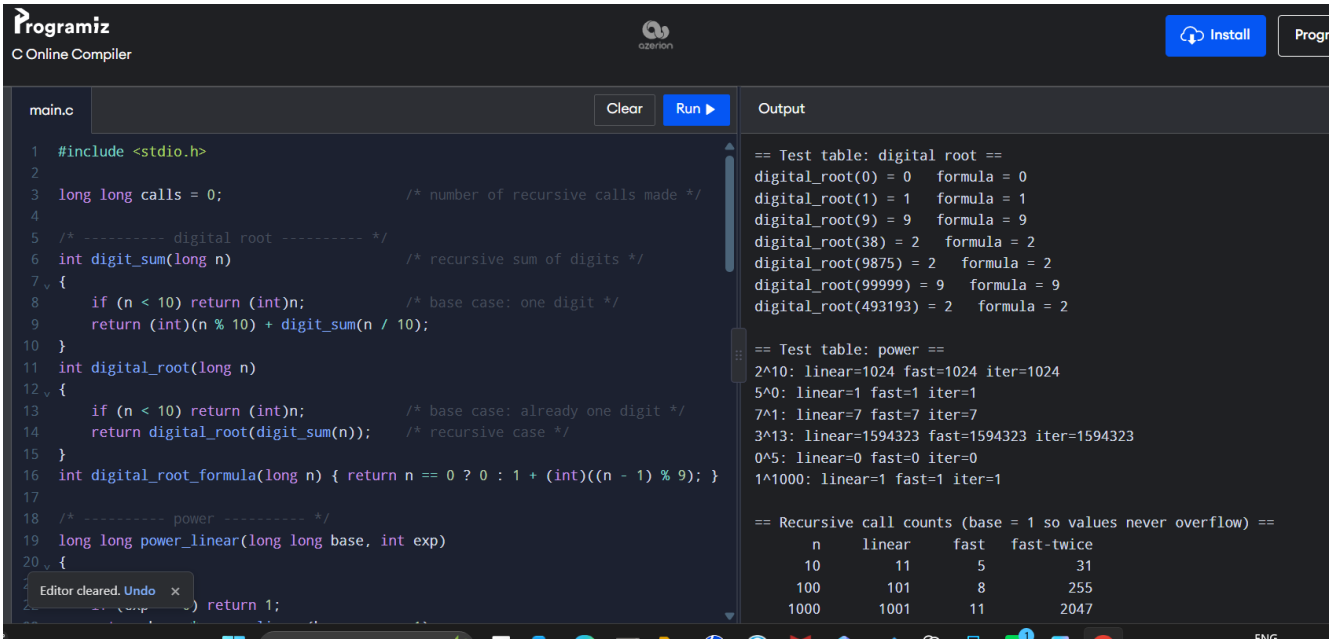
int main(void)
{
    calls = 0;
    long long a = power_linear(2, 30);
    printf("linear: %lld in %lld calls\n", a, calls);
    calls = 0;
    long long b = power_fast(2, 30);
    printf("fast : %lld in %lld calls\n", b, calls);
    return 0;
}
```

Program 3 – stack-limit program (b1_limit.c):

```
#include <stdio.h>
#include <stdlib.h>
```

```
long long power_linear(long long base, long exp) { if (exp == 0) return 1; return base * power_linear(base, exp - 1); }
long long power_iter(long long base, long exp) { long long r = 1; while (exp-- > 0) r *= base; return r; }
int main(int argc, char **argv)
{
    long n = atol(argv[1]);
    long long v = (argc > 2) ? power_iter(1, n) : power_linear(1, n);
    printf("%s n=%ld ok (result %lld)\n", argc > 2 ? "iterative" : "recursive", n, v);
    return 0;
}
```

Output Screenshot

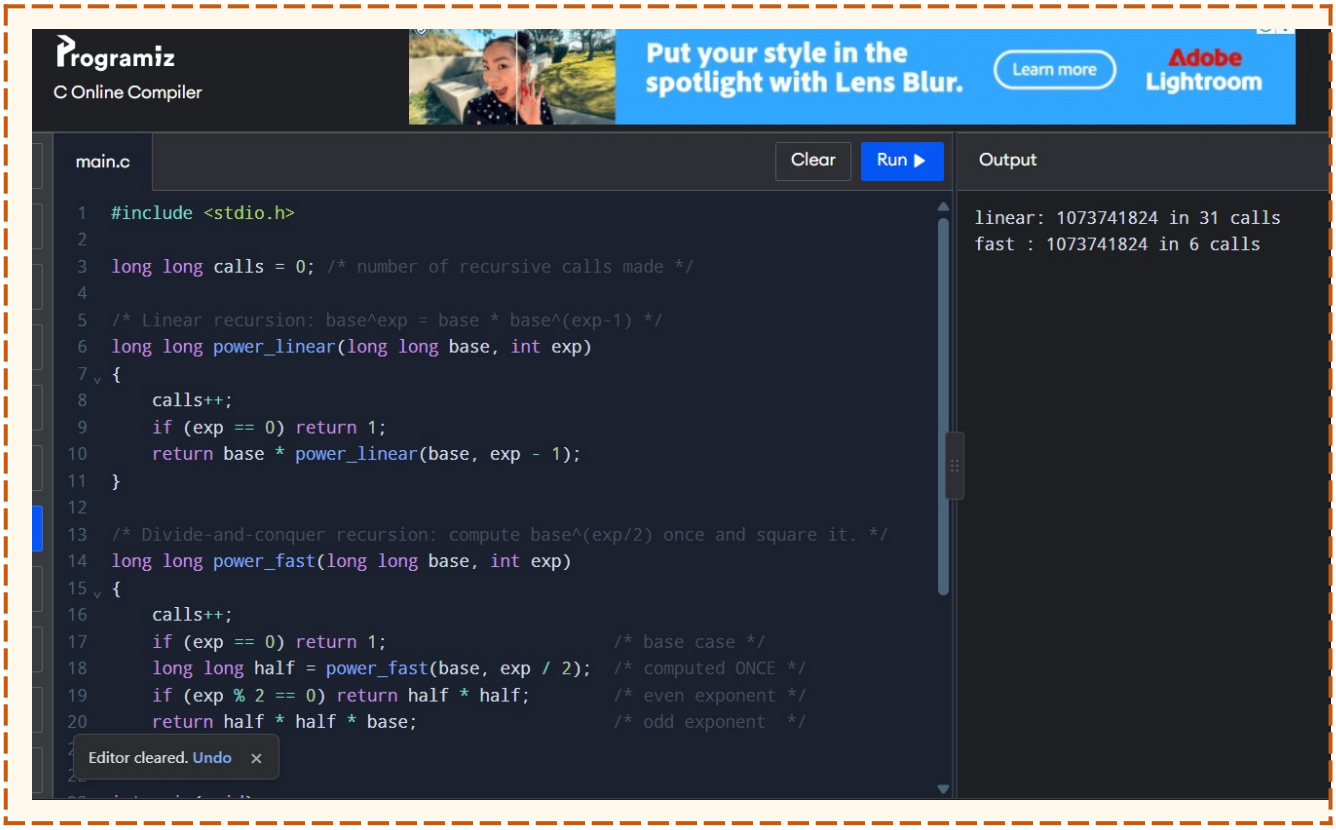



```
== Test table: digital root ==
digital_root(0) = 0   formula = 0
digital_root(1) = 1   formula = 1
digital_root(9) = 9   formula = 9
digital_root(38) = 2   formula = 2
digital_root(9875) = 2   formula = 2
digital_root(99999) = 9   formula = 9
digital_root(493193) = 2   formula = 2

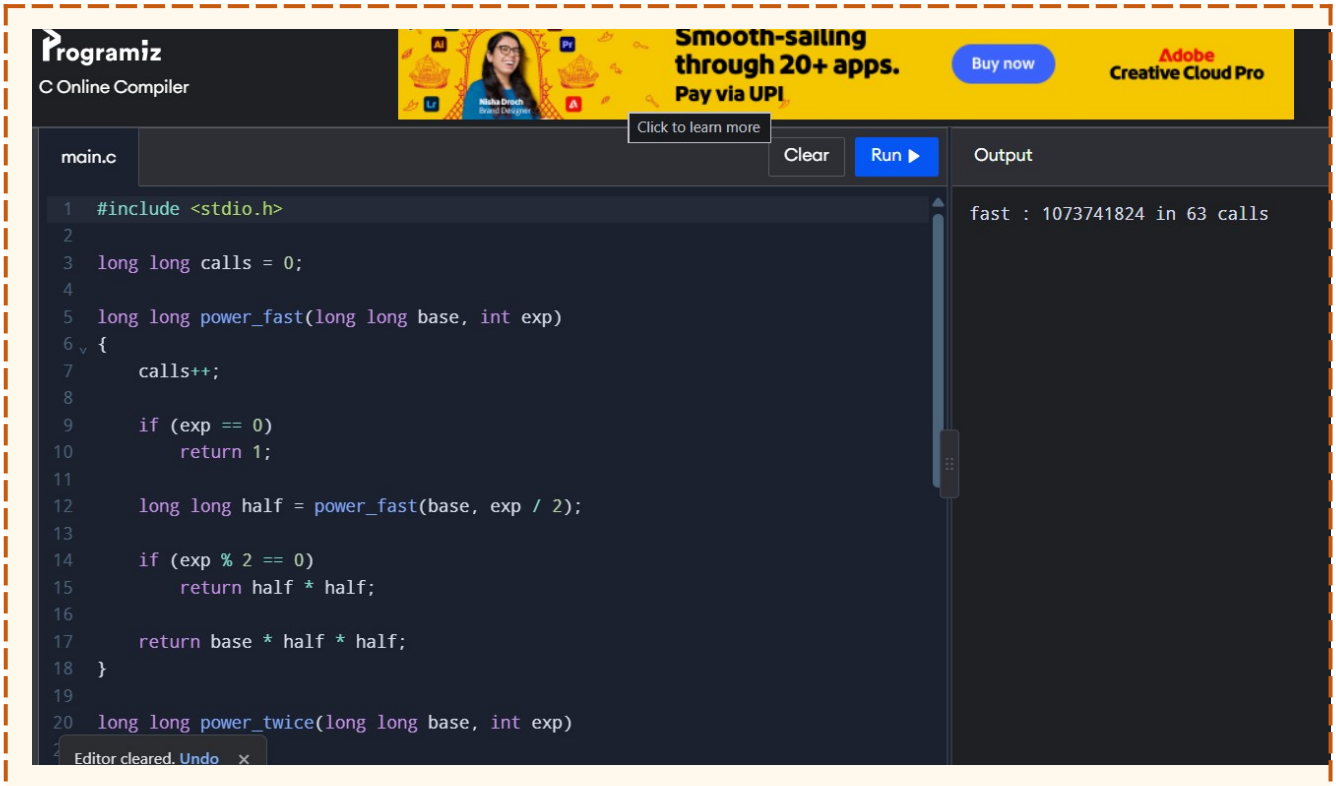
== Test table: power ==
2^10: linear=1024 fast=1024 iter=1024
5^0: linear=1 fast=1 iter=1
7^1: linear=7 fast=7 iter=7
3^13: linear=1594323 fast=1594323 iter=1594323
0^5: linear=0 fast=0 iter=0
1^1000: linear=1 fast=1 iter=1

== Recursive call counts (base = 1 so values never overflow) ==
      n     linear     fast   fast-twice
     10         11        5           31
    100        101        8          255
   1000       1001       11         2047
  10000      10001       15        32767

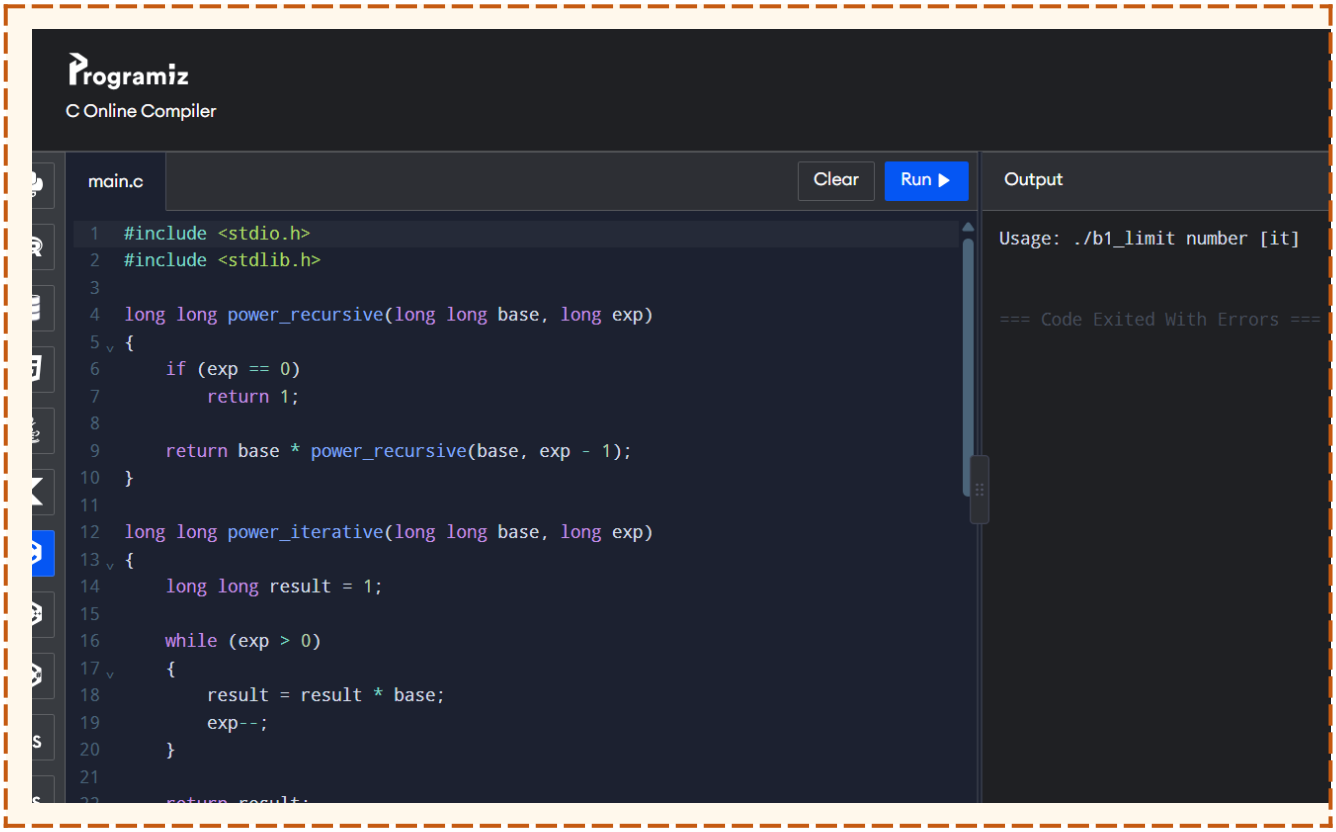
fast-twice for n = 30: 63 calls
```



linear: 1073741824 in 31 calls
fast : 1073741824 in 6 calls



```
linear: 1073741824 in 31 calls
fast : 1073741824 in 63 calls
```




```
$ ulimit -s
8192
$ setarch -R ./b1_limit 261401
recursive n=261401 ok (result 1)
$ setarch -R ./b1_limit 261402
Segmentation fault         (exit status 139)
$ ./b1_limit 2000000000 it
iterative n=2000000000 ok (result 1)
```

Analysis / Discussion

B1.1 – Validate the implementation (2 marks). The test table below shows what each test verifies.

Test	Input	Result	What it verifies
Ordinary value	digital_root(9875); 2^10; 3^13	2; 1024; 1,594,323	General case: several recursive steps, correct arithmetic; all three power versions agree
Zero	digital_root(0); 5^0; 0^5	0; 1; 0	Base case n < 10 / exp = 0 is reached immediately, and 0 as a base does not break the result
One	digital_root(1); 7^1; 1^1000	1; 7; 1	Smallest non-trivial exponent (one recursive step); base 1 is a stable fixed point
Multiple of 9	digital_root(9); 99999	9; 9	Checks that the root of a multiple of 9 is 9, not 0 (the formula's n - 1 trick)

Test	Input	Result	What it verifies
Own value	digital_root(493193); 3^13	2; 1,594,323	A 6-digit number with several rounds (493193 → 29 → 11 → 2); odd exponent for the halving version (13 is odd at three levels)
Formula cross-check	1 + (n−1) mod 9 for every digital-root input	same as recursive	Confirms the O(1) alternative of A2

B1.2 – Compare with the design (1 mark). The code follows the A2.1 pseudocode with these changes: (1) DigitSum and DigitalRoot are separate C functions exactly as designed, with the type long for the number; (2) half is stored in a variable so the recursive call is made once; (3) the odd-exponent case is half * half * base, the pseudocode's ELSE branch; (4) a global counter calls++ was added at the top of each power function – it is instrumentation only and is not part of the algorithm; (5) power_iter and digital_root_formula were added for the alternatives; (6) for the call-count experiment the base is 1, because 2^10000 does not fit in 64 bits and a signed overflow would hide the point of the test.

B1.3 – Count the calls (2 marks). Measured counts against the recurrences of A2.2:

n	Linear (measured)	n + 1	Halving (measured)	$\lfloor \log_2 n \rfloor + 2$	Two calls (measured)	$2^{(\lfloor \log_2 n \rfloor + 2)} - 1$
10	11	11	5	5	31	31
100	101	101	8	8	255	255
1,000	1,001	1,001	11	11	2,047	2,047
10,000	10,001	10,001	15	15	32,767	32,767

Every measured value equals its recurrence exactly. Going from n = 10 to n = 10,000 (1000 times larger), the linear version does 1000 times more work, the halving version only 3 times more.

B1.4 – Find the limit (2 marks). With the default 8 MB stack (ulimit -s = 8192 KB) and -O0, the linear version ran correctly for n = 261,401 and **crashed for n = 261,402** (segmentation fault). The stack holds one frame per call: 8,388,608 bytes ÷ 261,402 ≈ 32 bytes per frame (return address, saved frame pointer, base and exp). When the next call needs a frame beyond the stack's end, the program touches memory it does not own and the operating system stops it – a **stack overflow**. (The exact value changes by a few hundred between runs when address randomisation is on; it was fixed here with setarch -R.) The **iterative** version uses one frame and a loop, so it never crashed: it completed n = 2,000,000,000 (limited only by the range of int).

Explanation of the changed program. With power_fast(base, exp/2) * power_fast(base, exp/2) the same sub-problem is solved twice at every level. The recurrence changes from T(n) = 1 + T(n/2) to T(n) = 1 + 2T(n/2), whose solution is about 2n: 63 calls for n = 30 instead of 6, 32,767 instead of 15 for n = 10,000. The calculation is as slow (O(n)) as linear recursion, so the benefit of halving is lost. Computing half once is the essential part of the divide-and-conquer idea.

Alternative Concept and Complexity Analysis – iterative square-and-multiply

```
ALGORITHM PowerIter(base, n)
BEGIN
  result <- 1
  WHILE n > 0 DO
    IF n MOD 2 = 1 THEN result <- result * base     // current binary digit of n is 1
    base <- base * base ; n <- n DIV 2
  END WHILE
  RETURN result
END
```

Version	Time (multiplications / calls)	Stack space	Crash limit
Linear recursion	O(n)	O(n)	n ≈ 261,000 (8 MB stack)

Halving recursion	O(log n)	O(log n) – 15 frames for n = 10,000	n would have to exceed 2^(260,000) – impossible, exponent is an int
Iterative square-and-multiply	O(log n) – same number of steps as halving	**O(1)**	None

The iterative and the halving version do the same arithmetic and are both far faster than linear recursion; the iterative one additionally removes the stack cost and is the safest choice for production. All values above 2^63 overflow a long long; real reports would use a modulus or unsigned __int128.

Result

All three functions were implemented and gave correct results for ordinary, 0, 1 and own values. Call counts matched n + 1, $\lfloor \log_2 n \rfloor + 2$ and ~2n exactly; linear recursion overflowed the 8 MB stack at n = 261,402 while the iterative version did not.